

CPCS 301

Concepts of Programming Languages

Preliminaries



Dr. Asaad Ahmed
Department of Computer Science
Faculty of Computing and Information Technology
King Abdulaziz University

(Slides are adopted from *Concepts of Programming Languages*, R.W. Sebesta; *Modern Programming Languages: A Practical Introduction*, A.B. Webber)

Programming Language

- ◆ A programming language is a formal system used to instruct a computer
- ◆ It consists of a set of rules and symbols that allow humans to communicate instructions to a computer system
- ◆ These instructions, known as **code**, are written by programmers to perform specific tasks or solve particular problems

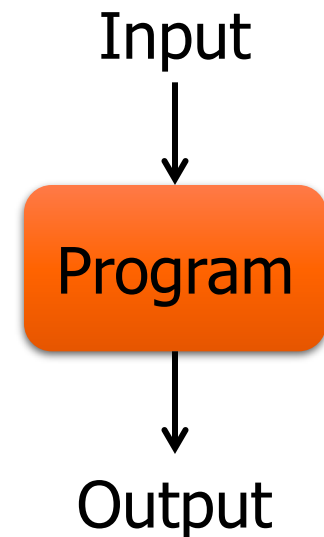
Programming Language

- ◆ Programming languages vary in their syntax, semantics, and purpose.
- ◆ Some languages are general-purpose, designed for a wide range of applications, while others are specialized for specific tasks
- ◆ A language used to write programs, where the program is instructions in computer memory to make it do something.
- ◆ A means of formal and exact communication between humans
- ◆ A way of describing computation that is readable by both man and machine

Programming Language

- ◆ A programming language is an artificial language designed to **express** computations or algorithms that can be performed by a computer
-- Wikipedia

- ◆ A program is computer coding of an algorithm that
 - Takes input
 - Performs some calculations on the input
 - Generates output



Reasons for Studying Concepts of Programming Languages



- ◆ Increased ability to express ideas
- ◆ Improved background for choosing appropriate languages
- ◆ Increased ability to learn new languages
- ◆ Better understanding of significance of implementation
- ◆ Overall advancement of computing

Increased ability to express ideas

- ◆ The depth at which people can think is influenced by the expressive power of the language in which they communicate their thoughts
- ◆ Programmers, in the process of developing software, are similarly constrained
- ◆ The language in which they develop software places limits on the kinds of control structures, data structures, and abstractions they can use; thus, the forms of algorithms they can construct are likewise limited
- ◆ Awareness of a wider variety of programming language features can reduce such limitations
 - **Example:** C programmer who had learned the structure and uses of associative arrays in Perl

Improved background for choosing appropriate languages



- ◆ Many programmers received their formal training years ago.
- ◆ The languages they learned then are no longer used, and many features now available in PLs were not widely known at the time.
- ◆ The result is that many programmers, they use the language with which they are most familiar, even if it is poorly suited for the project at hand.
- ◆ If these programmers were familiar with a wider range of languages, they would be better able to choose the language with the features that best address the problem.

Increased ability to learn new languages



- ◆ The process of learning a new programming language can be lengthy and difficult, especially for someone who is comfortable with only one or two languages and has never examined programming language concepts in general
- ◆ Once a thorough understanding of the fundamental concepts of languages is acquired, it becomes far easier to see how these concepts are incorporated into the design of the language being learned.
- ◆ For example, programmers who understand the concepts of object-oriented programming will have a much easier time learning Ruby

Better understanding of significance of implementation



- ◆ Understanding of implementation issues leads to an understanding of why languages are designed the way they are
- ◆ In turn, this knowledge leads to the ability to use a language more intelligently, as it was designed to be used
- ◆ Certain kinds of program bugs can be found and fixed only by a programmer who knows some related implementation details
 - E.g., passing by value and passing by reference
- ◆ Understanding implementation issues is that it allows us to visualize how a computer executes various language constructs
 - > E.g., stack (last in first out)

Better use of languages that already known



- ◆ Most current programming languages are large and complex.
- ◆ Accordingly, it is uncommon for a programmer to be familiar with and use all of the features of a language he or she uses.
- ◆ This course leads programmers to learn about previously unknown and unused parts of the languages they already use and begin to use those features.

Overall advancement of computing



- ◆ It is usually possible to determine why a particular programming language became popular, many believe, at least in retrospect, that the most popular languages are not always the best available
- ◆ In some cases, it might be concluded that a language became widely used, at least in part, because those in positions to choose languages were not sufficiently familiar with the features of other

Programming Domains

♦ Scientific applications

- Appeared in the late 1940s and early 1950s
- Large numbers of floating-point computations.
- The most common data structures were arrays and matrices;
- The most common control structures were counting loops and selections
- Fortran, ALGOL 60

♦ Business applications

- Appeared in the began of 1950s.
- Produce reports, use decimal numbers and characters
- Business languages are characterized by facilities for producing elaborate reports, precise ways of describing and storing decimal numbers and character data, and the ability to specify decimal arithmetic operations.
- COBOL

Programming Domains

◆ Artificial intelligence

- Symbols rather than numbers manipulated
- Use of linked lists - LISP, Prolog
- Now the new AI systems written using MATLAB, Python, R, etc.
- LISP

◆ Systems programming

- Need efficiency because of continuous use
- C

◆ Web Software

- Eclectic collection of languages: markup (e.g., XHTML), scripting (e.g., PHP), general-purpose (e.g., Java)

◆ Mobile applications

Language Evaluation Criteria

- ◆ **Readability:** the ease with which programs can be read and understood
- ◆ **Writability:** the ease with which a language can be used to create programs
- ◆ **Reliability:** conformance to specifications (i.e., performs to its specifications)
- ◆ **Cost:** the ultimate total cost

Evaluation Criteria: Readability



- ◆ Overall simplicity
- ◆ Orthogonality
- ◆ Control statements
- ◆ Data types and structures
- ◆ Syntax considerations

Evaluation Criteria: Readability

◆ Overall simplicity

● A manageable set of features and constructs

- The overall simplicity of a programming language strongly affects its readability
- A language with a large number of basic constructs is more difficult to learn than one with a smaller number
- Programmers who must use a large language often learn a subset of the language and ignore its other features
- Readability problems occur whenever the program's author has learned a different subset from that subset with which the reader is familiar

Evaluation Criteria: Readability

◆ Overall simplicity

● Minimal feature multiplicity

- More than one way to accomplish a particular operation
- Example in Java

```
count = count + 1
```

```
count += 1
```

```
count++
```

```
++count
```

● Minimal operator overloading

- A single operator symbol has more than one meaning
- Example in C++

```
count = 4
```

```
if(count = 5)
```

Evaluation Criteria: Readability

◆ Orthogonality

- A relatively small set of primitive constructs can be combined in a relatively small number of ways to build the control and data structures of the language.
- Every possible combination is legal
- Primitive data types (integer, float, double, and character).
- Two type operators (array and pointer)
- A lack of orthogonality leads to exceptions to the rules of the language.

Evaluation Criteria: Readability

◆ Control statements

- The presence of well-known control structures (e.g., while statement)

Evaluation Criteria: Readability

◆ Data types and structures

- The presence of adequate facilities for defining data structures
- Adequate predefined data types
- suppose a numeric type is used for an indicator flag because there is no Boolean type in the language

timeout = 1 (meaning of this statement is unclear)

timeout = true (meaning of this statement is clear)

Evaluation Criteria: Readability

◆ Syntax considerations

- Identifier forms: flexible composition
- Special words and methods of forming compound statements
- Form and meaning: self-descriptive constructs, meaningful keywords
 - For example, in Fortran 95, special words, such as Do and End, are legal variable names, so the appearance of these words in a program may or may not connote something special.
- Using words where their appearance at least partially indicates their purpose is an obvious aid to readability
 - For example, the same variable defined in the function as a local variable, and defined outside all functions as a global variable

Evaluation Criteria: Writability

- ◆ Writability is a measure of how easily a language can be used to create programs for a chosen problem domain
- ◆ It simply is not fair to compare the writability of two languages in the realm of a particular application when one was designed for that application and the other was not
- ◆ For example, the writabilities of Visual BASIC (VB) (Halvorson, 2013) and C are dramatically different for creating a program that has a graphical user interface (GUI), for which VB is ideal

Evaluation Criteria: Writability

◆ **Simplicity and orthogonality**

- Few constructs, a small number of primitives, a small set of rules for combining them
- If a language has a large number of different constructs, some programmers might not be familiar with all of them

◆ **Support for abstraction**

- The ability to define and use complex structures or operations in ways that allow details to be ignored

◆ **Expressivity**

- A set of relatively convenient ways of specifying operations
 - Example: the inclusion of **for statement** in many modern languages
- Strength and number of operators and predefined functions
 - E.g., notation `count++` is more convenient and shorter than `count = count + 1`

Evaluation Criteria: Reliability

A program is said to be **reliable** if it performs to its specifications under all conditions

◆ Type checking

- Type checking is simply testing for type errors in a given program, either by the compiler or during program execution
- Testing for type errors
- Run-time type checking is expensive, compile-time type checking is more desirable
- The earlier errors in programs are detected, the less expensive it is to make the required repairs
- Presence of two or more distinct referencing methods for the same memory location

Evaluation Criteria: Reliability

◆ Exception handling

- Intercept run-time errors and take corrective measures
- The ability of a program to intercept run-time errors
- Ada, C++, Java, and C# include extensive capabilities for exception handling, nonexistent in some widely used languages, for example C

◆ Aliasing

- Presence of two or more distinct referencing methods for the same memory location, E.g. pointers in C++, Array name in Java
- Other languages greatly restrict aliasing to increase their reliability

Evaluation Criteria: Reliability

◆ Readability and writability

- A language that does not support “natural” ways of expressing an algorithm will necessarily use “unnatural” approaches, and hence reduced reliability
- The easier a program is to write, the more likely it is to be correct
- Programs that are difficult to read are difficult both to write and to modify

Evaluation Criteria: Cost

The total cost of a programming language is a function of many of its characteristics

- ◆ **Training programmers to use language**
 - is a function of the simplicity and orthogonality
- ◆ **Writing programs (closeness to particular applications)**
 - is a function of the writability of the language
- ◆ **Compiling programs**

Evaluation Criteria: Cost

◆ Executing programs

- A language that requires many run-time type checks will prohibit fast code execution
- A simple trade-off can be made between compilation cost and execution speed of the compiled code

Evaluation Criteria: Cost

- ◆ **Language implementation system: availability of free compilers**
 - A language whose implementation system is either expensive or runs only on expensive hardware will have a much smaller chance of becoming widely used
- ◆ **Reliability: poor reliability leads to high costs**
 - E.g., critical systems
- ◆ **Maintaining programs**
 - The cost of software maintenance depends on a number of language characteristics, primarily readability

Evaluation Criteria: Others



◆ **Portability**

- The ease with which programs can be moved from one implementation to another

◆ **Generality**

- The applicability to a wide range of applications

◆ **Well-definedness**

- The completeness and precision of the language's official definition

Influences on Language Design

◆ Computer Architecture

- New software development methodologies (e.g., object-oriented software development) led to new programming paradigms and by extension, new programming languages

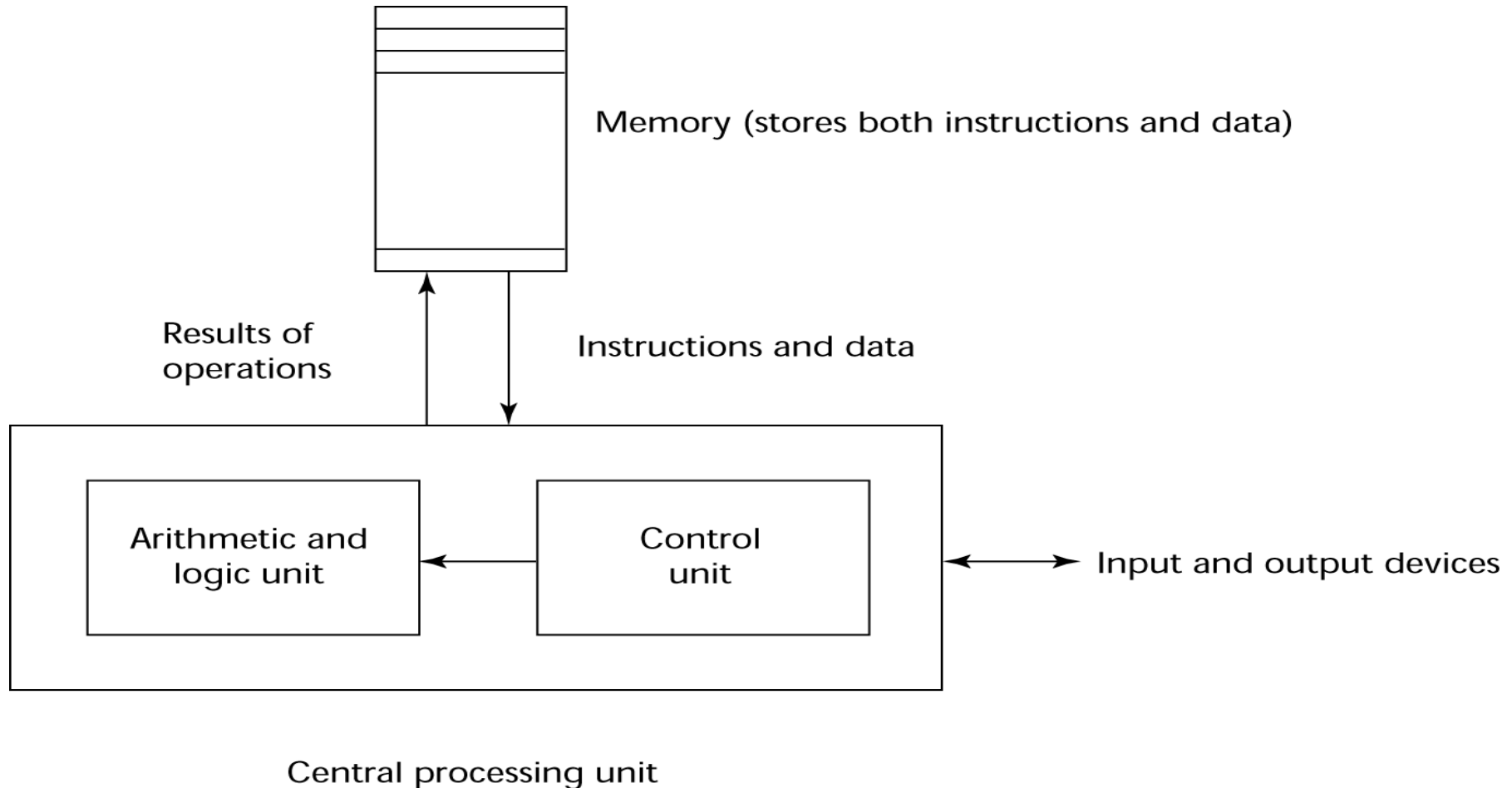
◆ Program Design Methodologies

- Languages are developed around the prevalent computer architecture, known as the **von Neumann architecture**

Computer Architecture Influence

- ◆ Well-known computer architecture: **Von Neumann**
- ◆ Imperative languages, most dominant, because of von Neumann computers
 - Data and programs stored in memory
 - Memory is separate from CPU
 - Instructions and data are piped from memory to CPU
 - Basis for imperative languages
 - Variables model memory cells
 - Assignment statements model piping
 - Iteration is efficient

The von Neumann Architecture



The von Neumann Architecture

◆ Fetch-execute-cycle (on a von Neumann architecture computer)

initialize the program counter

repeat forever

 fetch the instruction pointed by the counter

 increment the counter

 decode the instruction

 execute the instruction

end repeat

Programming Methodologies Influences

- ◆ 1950s and early 1960s: Simple applications; worry about machine efficiency
- ◆ Late 1960s: People efficiency became important; readability, better control structures
 - structured programming
 - top-down design and step-wise refinement
- ◆ Late 1970s: Process-oriented to data-oriented
 - data abstraction
 - The first language to provide even limited support for data abstraction was SIMULA 67
- ◆ Middle 1980s: Object-oriented programming
 - Data abstraction + inheritance + polymorphism

Models of Programming Languages



Programming is like ...

Language Categories

- ◆ Programming languages are often categorized into four bins:
 - Imperative
 - Functional
 - Logic
 - Markup/programming hybrid
- ◆ However, we do not consider languages that support object-oriented programming to form a separate category of languages
 - We have described how the most popular languages that support object-oriented programming grew out of imperative languages
 - For example, the expressions, assignment statements, and control statements of C and Java are nearly identical

Language Categories

◆ Imperative

- Central features are variables, assignment statements, and iteration
- Include languages that support object-oriented programming - Include scripting languages
- Include the visual languages
- Examples: C, Pascal, Java, Perl, JavaScript, Visual BASIC .NET, C++

◆ Functional

- Main means of making computations is by applying functions to given parameters
- Examples: LISP, Scheme, ML, F#

◆ Logic

- Rule-based (rules are specified in no particular order)
- Example: Prolog

Language Categories

◆ Object-oriented

- Data abstraction, inheritance, late binding
- Examples: Java, C++

◆ Markup/programming hybrid

- New; not a programming per se, but used to specify the layout of information in Web documents
- Markup languages are not programming languages.
- Markup languages extended to support some programming
- Examples: HTML, XHTML, XML, JSTL, XSLT

1st View: Imperative & Procedural

- ◆ Computers take commands and do operations
- ◆ Thus, programming is like ...
issuing procedural commands to the computer
 - Example: a factorial function in C

```
int fact(int n) {  
    int sofar = 1;  
    while (n>0) sofar *= n--;  
    return sofar;  
}
```
- ◆ Since almost all computers today use the von Neumann architecture → PL mimic the arch.

Imperative Languages and Arch.

- ◆ *Imperative languages*, e.g., C, C++, Java, which dominate programming, mimic von Neumann architecture
 - Variables $\leftrightarrow$ memory cells
 - Assignment statements $\leftrightarrow$ data piping between memory and CPU
 - Operations and expressions $\leftrightarrow$ CPU executions
 - Explicit control of execution flows $\leftrightarrow$ prog. counter
- ◆ Allow efficient mapping between language and hardware for good execution performance, but limited by von Neumann bottleneck

Layers of Abstraction/Translation

High Level Language Program

Compiler

Assembly Language Program

Assembler

Machine Language Program

Machine Interpretation

Control Signal Specification

```
temp = v[k];
```

```
v[k] = v[k+1];
```

```
v[k+1] = temp;
```

```
lw      $15, 0($2)
```

```
lw      $16, 4($2)
```

```
sw$16, 0($2)
```

```
sw$15, 4($2)
```

```
0000 1001 1100 0110 1010 1111 0101 1000
1010 1111 0101 1000 0000 1001 1100 0110
1100 0110 1010 1111 0101 1000 0000 1001
0101 1000 0000 1001 1100 0110 1010 1111
```

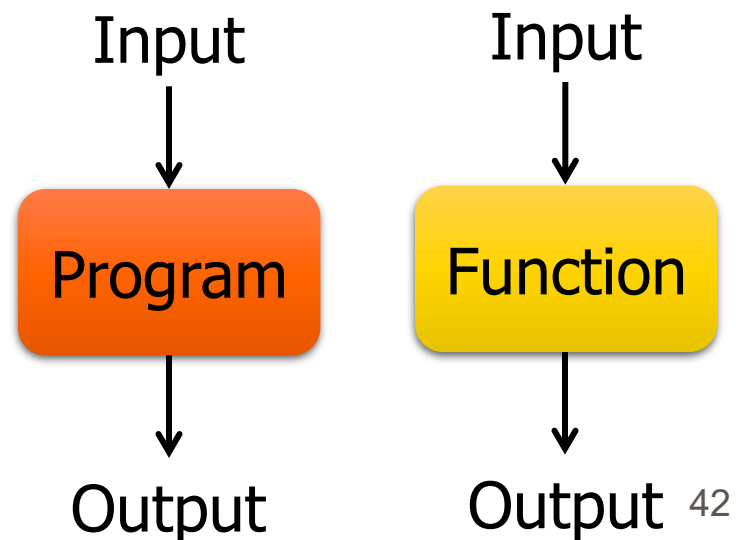
All imperative!

```
ALUOP[0:3] <= InstReg[9:11] & MASK
```

2nd View: Functional

- ◆ Programming is like ...
solving mathematical functions, e.g.,
 $z = f(y, g(h(x)))$
 - A program, and its subprograms, are just implementations of mathematical functions
 - Example: a factorial function in ML

```
fun fact x =  
  if x <= 0  
  then 1  
  else x * fact(x-1) ;
```



Another Functional Language: LISP

◆ Example: a factorial function in Lisp

```
(defun fact (x)
```

```
  (if (<= x 0) 1 (* x (fact (- x 1)))))
```

- Computations by applying functions to parameters
 - No concept of variables (storage) or assignment
 - Single-valued variables: no assignment, not storage
 - Control via recursion and conditional expressions
 - Branches → conditional expressions
 - Iterations → recursion
 - Dynamically allocated linked lists
- ## ◆ 2nd-oldest general-purpose PL still in use (1958)

3rd View: Logic

◆ Programming is like ...

logic induction

- Program expressed as rules in formal logic
- Execution by rule resolution
- Example: relationship among people

```
fact:  mother(joanne,jake) .  
       father(vern,joanne) .
```

```
rule:  grandparent(X,Z) :- parent(X,Y) ,  
                           parent(Y,Z) .
```

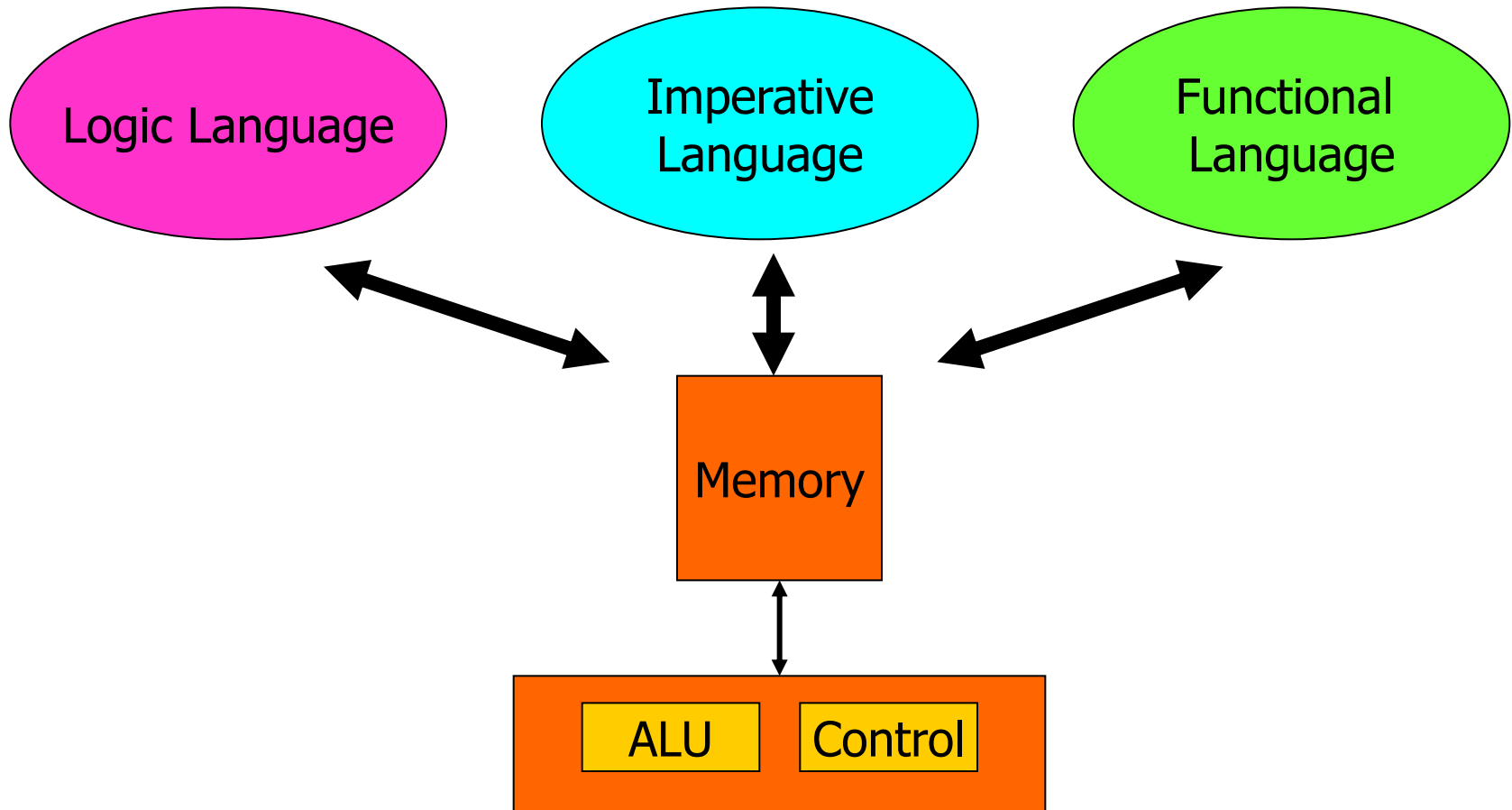
```
goal:  grandparent(vern,jake) .
```

Logic Programming

- ◆ Non-procedural
 - Only supply relevant facts (predicate calculus) and inference rules (resolutions)
 - System then infer the truth of given queries/goals
- ◆ Highly inefficient, small application areas (database, AI)
 - Example: a factorial function in Prolog

```
fact(X,1) :- X == 1.  
fact(X,Fact) :-  
    X > 1, NewX is X - 1,  
    fact(NewX,NF) ,  
    Fact is X * NF.
```

Summary: Language Categories



von Neumann Architecture

Summary: Language Categories

◆ Imperative

- Variables, assignment statements, and iteration
- Include languages that support object-oriented programming, scripting languages, visual languages
- Ex.: C, Java, Perl, JavaScript, Visual BASIC .NET

◆ Functional

- Computing by applying functions to given parameters
- Ex.: LISP, Scheme, ML

◆ Logic

- Rule-based (rules are specified in no particular order)
- Ex.: Prolog

Language Design Trade-Offs

◆ Reliability vs. cost of execution

- Conflicting criteria
- Example: Java demands all references to array elements be checked for proper indexing but that leads to increased execution costs

◆ Readability vs. writability

- Another conflicting criteria
- Example: APL provides many powerful operators (and a large number of new symbols), allowing complex computations to be written in a compact program but at the cost of poor readability

◆ Writability (flexibility) vs. reliability

- Another conflicting criteria
- Example: C++ pointers are powerful and very flexible but not reliably used

Implementation Methods



- ◆ Compilation
 - Programs are translated into machine language
- ◆ Pure Interpretation
 - Programs are interpreted by another program known as an interpreter
- ◆ Hybrid Implementation Systems
 - A compromise between compilers and pure interpreters

Compilation

- ◆ Translate high-level program (source language) into machine code (machine language)
- ◆ Slow translation, fast execution
- ◆ Compilation process has several phases:
 - **lexical analysis**: converts characters in the source program into lexical units
 - **syntax analysis**: transforms lexical units into *parse trees* which represent the syntactic structure of program
 - **Semantics analysis**: generate intermediate code
 - **code generation**: machine code is generated

Compilation



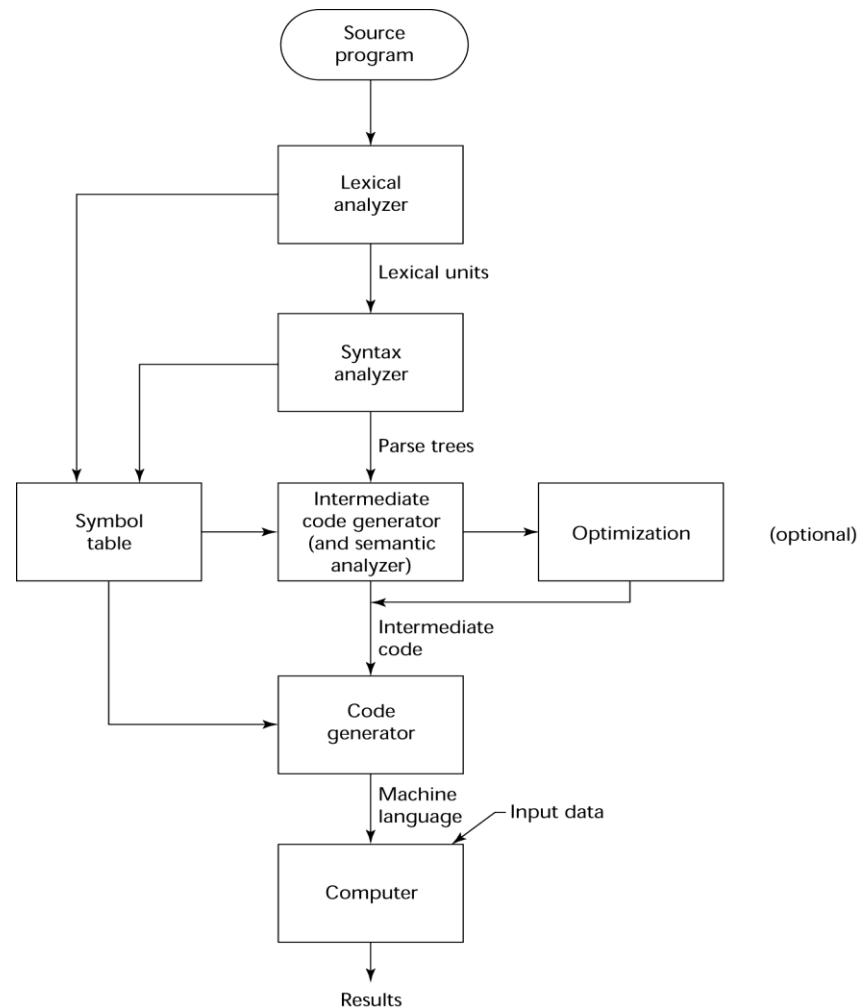
Example Program

```
int sum(int a, int b) {  
    return a + b;  
}
```

Compilation

	Lexical Analysis	Syntax Analysis	Semantic Analysis	Code Generation
input	The source code	Sequence of tokens	Parse tree	Intermediate code
output	Tokens generated int, sum (int a , int B) { Return a + b ; }	Parse Tree ReturnType: int FunctionName: sum Parameters Parameter: int a Parameter: int b Body ReturnStatement BinaryExpression Identifier: a Operator: + Identifier: b	Intermediate Code t1 = a + b return t1	Machine Code LOAD R1, a LOAD R2, b ADD R3, R1, R2 STORE R3, result RETURN

The Compilation Process



Additional Compilation Terminologies

- ◆ **Load module** (executable image): the user and system code together
- ◆ **Linking and loading**: the process of collecting system program and linking them to user program

Execution of Machine Code

◆ Fetch-execute-cycle (on a von Neumann architecture)

```
initialize the program counter
```

```
repeat forever
```

```
    fetch the instruction pointed by the counter
```

```
    increment the counter
```

```
    decode the instruction
```

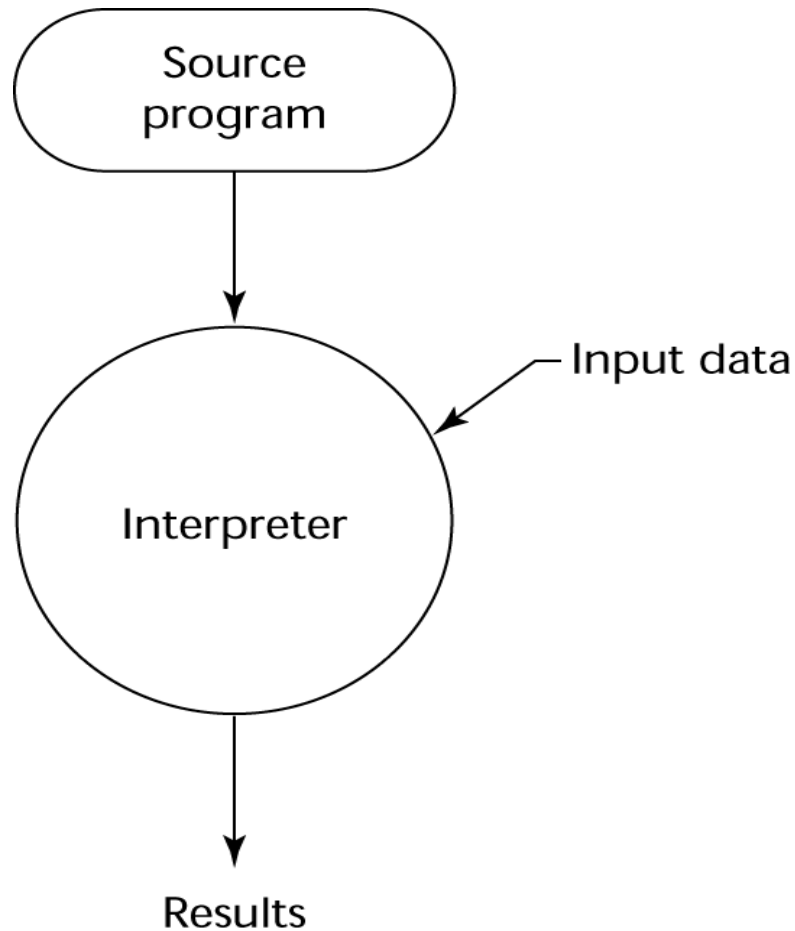
```
    execute the instruction
```

```
end repeat
```


Pure Interpretation

- ◆ No translation
- ◆ Easier implementation of programs (run-time errors can easily and immediately be displayed)
- ◆ Slower execution (10 to 100 times slower than compiled programs)
- ◆ Often requires more space
- ◆ Becoming rare on high-level languages
- ◆ Significant comeback with some Web scripting languages (e.g., JavaScript)

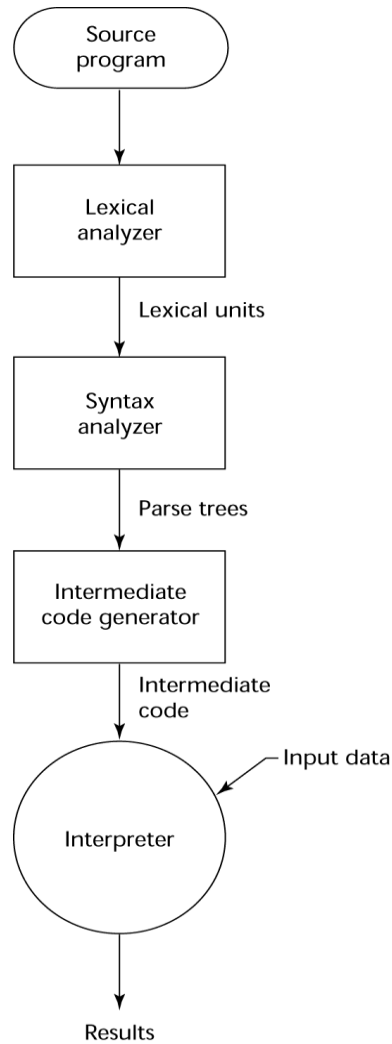
Pure Interpretation Process



Hybrid Implementation Systems

- ◆ A compromise between compilers and pure interpreters
- ◆ A high-level language program is translated to an intermediate language that allows easy interpretation
- ◆ Faster than pure interpretation
- ◆ Examples
 - Perl programs are partially compiled to detect errors before interpretation
 - Initial implementations of Java were hybrid; the intermediate form, *byte code*, provides portability to any machine that has a byte code interpreter and a run-time system (together, these are called *Java Virtual Machine*)

Hybrid Implementation Process



Programming Environments

- ◆ The collection of tools used in software development
- ◆ UNIX
 - An older operating system and tool collection
 - Nowadays often used through a GUI (e.g., CDE, KDE, or GNOME) that run on top of UNIX
- ◆ Borland JBuilder
 - An integrated development environment for Java
- ◆ Microsoft Visual Studio.NET
 - A large, complex visual environment
 - Used to program in C#, Visual BASIC.NET, Jscript, J#, or C++

Summary



- ◆ The study of programming languages is valuable for a number of reasons:
 - Increase our capacity to use different constructs
 - Enable us to choose languages more intelligently
 - Makes learning new languages easier
- ◆ Most important criteria for evaluating programming languages include:
 - Readability, writability, reliability, cost
- ◆ Major influences on language design have been machine architecture and software development methodologies
- ◆ The major methods of implementing programming languages are: compilation, pure interpretation, and hybrid implementation